The React edition

How react.mvcprogrammer.com is built, how it differs from the vanilla edition at mvcprogrammer.com, the Angular edition at angular.mvcprogrammer.com, and the Blazor edition at blazor.mvcprogrammer.com, and the rules for changing, testing, and deploying it.

1. Why there are four editions

The portfolio ships four times. The default edition is plain HTML, CSS, and JavaScript at the repository root, served at mvcprogrammer.com. The Blazor edition is a Blazor WebAssembly app under `blazor/`, served at blazor.mvcprogrammer.com and documented in docs/BLAZOR-EDITION.pdf. The Angular edition is an Angular 22 workspace under `angular/`, served at angular.mvcprogrammer.com and documented in docs/ANGULAR-EDITION.pdf. The React edition is a React 19 application built with Vite under `react/`, served at react.mvcprogrammer.com. All four show the same content, load the same stylesheet under their own palettes, and call the same Experience API. The React edition exists to show that content consumed from function components and hooks, next to the Angular, C#, and vanilla versions.

The one rule that matters

Every content, markup, behavior, or styling change made to one edition is made to the other three in the same commit. Before finishing any change to the site, check the other editions for the same change. Section 4 lists which files are shared (one edit) and which are duplicated (four edits).

No exceptions

Content is identical on every edition. The editions cross-link through the About core-skill chips (**Angular**, **React**, **Blazor**, **JavaScript**, each with a title hint such as See the React edition of this site) and through the edition menu in the header: a pill naming the open edition that lists all four. The hero highlights are plain text (their Angular and React links were removed on 2026-10-19). A link to the edition already open simply reloads it. The portfolio card carries no edition links. The test in `sections/static-sections.test.tsx` pins the React side of it.

The one exception: color

Each edition carries its own palette. One small file per edition, loaded after the shared `styles.css`, overrides only the design tokens (`--bg`, `--accent`, `--hover`, and the rest) with `light-dark()` values, so one declaration covers both color schemes and no rule from the shared stylesheet is repeated. Older browsers ignore `light-dark()` and fall back to the shared JetBrains Rider palette. The four palettes, with the page, the accent, and the hover color for each scheme:

Edition	Palette file	Palette	Light	Dark
Default	`vanilla.css`	Eggshell paper with a terracotta accent and a coral hover; a warm charcoal after dark.		
Angular	`angular/src/angular.css`	Navy and brass: a cool, faintly blue page with navy buttons and a goldenrod hover; a true navy after dark with muted brass for buttons and links and a paler gold on hover.		
React	`react/src/react.css`	The react.dev documentation palette: cool slate surfaces, React blue for buttons and links, the logo cyan on hover.		
Blazor	`blazor/src/Portfolio.Blazor/wwwroot/blazor.css`	.NET violet: the .NET brand purple on faint lavender, a deep violet-black with lilac buttons after dark, the Blazor gradient's pink on hover.		

Vite's CSS minifier would rewrite `light-dark()` into a variable polyfill that only works when `color-scheme` is declared in the same file, and it is declared in the shared `styles.css`, so `vite.config.ts` pins `build.cssTarget` to browsers that support the function and the minifier leaves it alone. Check the production stylesheet under `dist/bundles/` after any change to the palette, not only the dev server.

Every edition's colophon describes its own palette, and the four are listed with screenshots in the README of this repository and on the GitHub profile README.

2. Where things live

The project follows the folder-by-kind layout from the coding standards, in React's dress: plural folder names under `src/app/`, PascalCase files for components and kebab-case files for everything else, and one `*.test.ts(x)` beside each source file.

Path	What it holds
`react/`	The project: one Vite application, `portfolio-react`, React 19, strict TypeScript, Vitest with Testing Library.
`scripts/sync-shared.mjs`	Copies the shared root files into `public/` before every build and serve (the `prebuild` and `prestart` npm hooks). `public/` is generated and not committed.
`index.html`	The shell: metadata, the shared stylesheet link, the boot screen inside `#root`, and the module script tag Vite rewrites.
`vite.config.ts`	The React plugin, `assetsDir: 'bundles'`, `assetsInlineLimit: 0`, `cssTarget` (keeps `light-dark()` in the palette, section 1), and the Vitest section (jsdom, globals, the setup file).

Path	What it holds
src/main.tsx	The siteOptions literal, createSiteServices(), and createRoot().render() under StrictMode. Imports react.css.
src/react.css	The only extra stylesheet: this edition's palette (section 1), the boot screen, and the aria-invalid validation state.
src/app/App.tsx	App (SiteProvider, BrowserRouter, Layout) and Layout: skip link, icon sprite, header, <main> with the routes, footer.
pages/	Home (the sections in page order; starts the engagement tracker in an effect) and NotFound.
sections/	One component per section of the page: Hero, About, Quote, Services, Portfolio, Resume, Contact, Colophon. JSX lifted from index.html.
components/	Header (menu, scroll state, current section), footer, icon and icon sprite, social links, the experience timeline with its skill filter, and the kiosk demo dialog.
configurations/	SiteOptions.
interfaces/	ExperienceClient, ContactClient, ContactTopicState, EngagementTracker: the contracts the components depend on, so tests swap in fakes.
services/	FetchExperienceClient and FetchContactClient over fetch with AbortSignal.timeout, ListenerContactTopicState, and BeaconEngagementTracker.
providers/	createSiteServices() (validates the options, builds the services) and SiteProvider with the useSite() hook. The counterpart of AddPortfolioServices() and provideSite().
models/, requests/	Interfaces matching api/openapi.yaml and the contact Lambda's JSON body.
enumerations/	WorkMode, a const object and union type matching the API's kebab-case values.
exceptions/	HttpStatusError, thrown by the clients for a non-2xx response, carrying the status.
formatters/	monthLabel (2023-12 to Dec 2023) and placeLabel (Remote, Hybrid, or the place).
validators/	validateContactForm() with the length rules and the phone-required-for-call rule, plus REPLY_BY_PHONE.
src/testing/	Fakes of the clients and the tracker, fixture factories, the test options, renderWithSite(), and DOM helpers. Section 6.
react/README.md	The short version of this document, kept in the repo.

The project is not in mvcprogrammer.com.sln; open react/ as its own project in the IDE.

3. How the React edition differs from the others

The visible page is the same. The differences are in how it gets there. The Blazor column is in the Blazor guideline.

Concern	Vanilla edition (root)	Angular edition (angular/)	React edition (react/)
Rendering	Hand-written HTML; reads fully without JavaScript.	Standalone components, OnPush, zoneless; a boot screen until the bundle runs.	Function components rendered by `createRoot` under `StrictMode`; the same boot screen until the 97 kB (compressed) bundle runs.
Where state lives	DOM attributes and closures in `main.js`.	Signals on the components; `computed()` derives labels and paths.	`useState` on the components; labels and paths are derived in the render. The kiosk dialog's open state is lifted to the portfolio section.
JavaScript	`main.js`, about 660 lines.	None outside the components; `DestroyRef` and effect cleanups.	None outside the components. `useEffect` attaches `IntersectionObserver`, `matchMedia`, and the document listeners and returns their teardown.
Experience timeline	In the HTML; re-rendered from the API when it answers.	`ExperienceTimeline` loads from the API, falls back to the seed, wrapped in `PendingTasks`.	Same component and same fallback in one `useEffect` with an `active` flag, so a response after unmount (or StrictMode's first run) is dropped.
Skill chips	Pinned, rest, demoted; twelve before the more button.	Same constants as statics on `ExperienceTimeline`.	Same constants exported from `ExperienceTimeline.tsx` with `orderChips()`. Pinning a different skill means editing all four.
Contact form	HTML5 validation plus `checkValidity()`; fetch.	A typed `FormBuilder` group; the phone control is enabled only for a call.	Controlled fields in one `useState`, `validateContactForm()` on every render, `aria-invalid` on fields the visitor left or a submit found invalid. Same status messages, including the timeout one.
Topic preselection	Buttons set the select's value.	`ContactTopicState`, a Subject the form subscribes to.	`ContactTopicState` with `subscribe()` returning an unsubscribe function; the form subscribes in an effect and returns it.
Configuration	`data-` attributes in the HTML.	A `SiteOptions` literal validated by `provideSite()`, injected through a token.	A `SiteOptions` literal in `main.tsx`, validated by `createSiteServices()`, supplied by `SiteProvider` and read with `useSite()`.
Header behavior	Listeners in `main.js`.	Observers in `SiteHeader`; re-attached after `NavigationEnd`.	Observers in `SiteHeader` effects; the section observer depends on `useLocation().pathname` so it re-attaches after a navigation.
Theme toggle	A click handler in `main.js` sets `data-theme` and saves the choice.	A `toggleTheme()` method on `SiteHeader`.	An exported `toggleTheme()` function used as the button's `onClick`. All four keep no theme state; the stylesheet picks the icon, and the shared `theme.js` in the head applies a saved choice before the first render.

Concern	Vanilla edition (root)	Angular edition (angular/)	React edition (react/)
Kiosk demo	Native dialog; iframe cleared on close.	`KioskDialog` calls `showModal()`; iframe while the `open` signal is true.	`KioskDialog` takes `open` and `onClose` props; an effect calls `showModal()` or `close()`, the dialog's `close` event reports back, the iframe exists only while open.
Styling	Shared stylesheet plus `vanilla.css` (palette).	Shared stylesheet plus `angular.css` (palette, boot screen); no component styles (CSP).	Shared stylesheet plus `react.css` (palette, boot screen). **No CSS-in-JS and no `style` attribute**: the CSP forbids inline styles. No host elements, so no `display: contents` list to maintain.
Build output	The files as committed.	`ng build`: esbuild, fingerprinted bundles, critical CSS not inlined.	`tsc -b` then `vite build`: one fingerprinted script and stylesheet under `bundles/`, nothing inlined, no source maps.
Hosting	Bucket root, distribution for mvcprogrammer.com.	`angular/` prefix, own distribution, S3 403 rewritten to `index.html`.	`react/` prefix, own distribution, the static site's response headers policy unchanged, S3 403 rewritten to `index.html` so React Router answers deep links.
Tests	None for the static page.	140 Vitest cases on jsdom.	188 Vitest cases on jsdom with Testing Library, run before every build.

4. Shared by copying versus duplicated by hand

Vite serves `public/` as-is, so the shared files are copied there by `scripts/sync-shared.mjs` before every build and serve, the same way the Angular edition does it, and `public/` stays out of git, so there is still one source on disk. Everything else is written once per edition.

Edit once (copied into public/)

- `styles.css`, the whole stylesheet, linked from `index.html` by hand, ahead of the build's own stylesheet. The palette on top of it is per edition (section 1), so a color change is four edits.
- `assets/**`: fonts (Inter and JetBrains Mono as variable WOFF2 subsets) and images. The bundles go under `bundles/` so the deploy can give the two folders different cache headers.
- `favicon.svg`, `favicon.ico`, `apple-touch-icon.png`.
- `api/seed/positions.json`, served to the React edition as `data/positions.json`.

Edit four times (one copy per edition)

- Markup: `index.html`, the Razor sections and components, the Angular templates, and the JSX under `react/src/app/sections/` and `react/src/app/components/`.
- Behavior: `main.js`, the Razor code-behinds plus `site.js`, the Angular component classes, and the React components and hooks.
- Constants that shape the page: pinned and demoted skills, the visible chip count, status messages, section ids and labels.

- Endpoints: `data-` attributes in `index.html`, `wwwroot/appsettings.json`, `app.config.ts`, and the `siteOptions` literal in `main.tsx`.

When a change touches a duplicated file, the commit is not finished until all four copies match. When the React copy changes behavior, add or update the test that proves it.

5. Running it locally

Node 24 and npm; the .NET SDK is not needed. All commands run from `react/`.

```
npm ci
npm start                       # http://localhost:5173, copies the shared files first
npm run build                   # dist/ (tsc -b, then vite build)
npm run preview                 # serves dist/ at http://localhost:4173
```

The dev server has no `/api`, so the timeline falls back to the bundled seed and the skill filter stays hidden. To see the filter, serve the build output behind the real CloudFront behaviors or any host that proxies `/api` to the Experience API. The dev server injects its own inline scripts for hot reloading, which is fine locally; the production build has none.

The contact Lambda rejects Origins it does not know, so the form cannot send from a local preview in any edition.

6. Testing

The suite is Vitest 5 on jsdom with React Testing Library, using the Vitest globals. Components render under a `SiteProvider` holding fakes of the client interfaces (`renderWithSite()` adds a memory router), so the skill filter, the seed fallback, the contact form's validation and status messages, the mobile menu, the section observer, and the kiosk dialog are all exercised without a browser. The two fetch clients are tested against a stubbed `fetch`. The deploy workflow runs the suite and `tsc -b` before every build, so a red test blocks the React deploy.

Commands

```
# run the suite once
npm test
# re-run on change
npm run test:watch
# tests whose name matches (a regular expression)
npx vitest run -t "ContactSection"
npx vitest run -t "should fall back to the seed"
# one file
npx vitest run src/app/components/SiteHeader.test.tsx
# type-check the app, the tests, and the Vite config
npm run typecheck
```

Conventions the suite follows

- One `*.test.ts` or `*.test.tsx` beside each source file. `describe` names the component or function, a nested `describe` names the method or concern, and `it` states the behavior in a sentence, so the report reads like the C# names `Method_ShouldExpectedResult`.

- Arrange, Act, Assert, each marked with a comment. Scenario setup across the fakes lives in `arrange...` helpers at the bottom of the file; fixtures come from `create...` factories in `src/testing/test-data.ts`.
- Dependencies of the component under test are faked, never the component itself. `createSiteTestServices()` builds the fakes, the real topic state, and the test options in one call.
- Async rules: a state change after a promise is awaited with `findBy...` or `waitFor`; a call into the component from outside React (the topic state, an observer callback) is wrapped in `act()`.
- jsdom has no `IntersectionObserver`, no `matchMedia`, and no modal dialogs. The components guard for that; the tests stub the observer and the dialog methods where the behavior matters, and query a closed dialog's button with `hidden: true`.

Test layout

Files mirror the app. The counts are test cases as Vitest reports them (256 in all).

File	Tests	What it proves
components/ExperienceTimeline.test.tsx	19	API render, date and place labels, Present, every fallback path (error, empty, not a list, seed failure, empty seed, skills failure, no skills), late response after unmount, chip order, the more button, chip titles, filtering, restore without refetch, filter failure, the loading class.
components/SiteHeader.test.tsx	26	Nav links and initial state, the theme toggle (both OS schemes, a saved choice both ways, no matchMedia, blocked storage, icons and name), the edition list (four entries, current marked, open and close, Escape, outside click, listener release), menu toggle, close on nav click, Escape with focus return, outside and inside clicks, the breakpoint, listeners released, scrolled class, current link set, cleared near the top, kept further down, no sections, no observer in the browser, disconnect on unmount, replaced after navigation.
components/KioskDialog.test.tsx	8	No iframe while closed, open shows the configured kiosk modally, showModal not repeated, close button, close while not open, backdrop versus inside click, parent closing it, Escape through the close event.
components/IconSprite.test.tsx	14	Every symbol the page references exists in the sprite; the sprite is hidden.
components/Icon, SocialLinks, SiteFooter	3	Icon references a sprite symbol, three social profiles, footer year and legal links.
sections/ContactSection.test.tsx	25	Phone field hidden by default, reveal and hide, topic preselection and unsubscribe, post and both confirmations with reset, honeypot forwarded, disabled while sending, 429, generic failure, timeout, non-HTTP error, values kept after failure, no post when invalid, when a call has no number, or the message is too short, blur marks a field, fixing clears it, and the message table.
sections/static-sections.test.tsx	7	Hero with plain highlights, the four About chips, the quote, the portfolio links and kiosk button, the iframe removed on close, the resume URL and education, the colophon.

File	Tests	What it proves
sections/call-to-action.test.tsx	1	The Services button selects its contact topic.
pages/Home, pages/NotFound, App.test.tsx	6	Home renders every section in order and starts and stops the tracker, NotFound, the layout around the routed page, an unknown path routes to NotFound, App under the browser router.
providers/site-services.test.ts, SiteProvider.test.tsx	14	Every service built, invalid options rejected before building, each required key, each URL key, every problem named at once, the provider hands services down, useSite throws without one.
services/fetch-experience-client.test.ts	12	Paths, Accept header and timeout signal, query-string escaping, blank skill ignored, error status, non-JSON body, null bodies, skills, seed, the global fetch by default, a rejected fetch propagated.
services/fetch-contact-client.test.ts	5	JSON post with the headers and timeout signal, empty success body, status code on rejection, timeout propagated, the global fetch by default.
services/beacon-engagement-tracker.test.ts	14	Load ping with screen class and color scheme, view pings once per section, click pings for edition and profile links with their place, leave pings with the visible seconds accumulated across hide and show, silence on localhost, no observer in the browser, failed pings ignored, restart, stop, stop before start.
services/listener-contact-topic-state.test.ts	5	Notify every subscriber, no subscribers, no replay, unsubscribe stops notifications, unsubscribe during a select.
formatters/, validators/, exceptions/	33	Month labels and unparseable input, place label per work mode, every contact form rule in every reply state, the HttpStatusError shape.
src/testing/		`FakeExperienceClient`, `FakeContactClient`, and `FakeEngagementTracker` (scriptable, record their calls), `createSiteTestServices()`, `renderWithSite()`, `TEST_SITE_OPTIONS`, fixture factories, `jsonResponse()`, and `textOf()`.

7. Deploying

The `deploy-react` job in .github/workflows/deploy.yml runs on every push to `main`, in parallel with the other three jobs. It sets up Node 24 with the npm cache, runs `npm ci`, `npm test`, and `npm run build` (which copies the shared files and type-checks first), assumes the deploy role through OIDC, and syncs the output in two passes before invalidating the distribution.

- The fingerprinted bundles under `bundles/` get a one-week browser TTL and `immutable`.
- Everything else, `index.html`, the shared stylesheet, icons, images, and the seed, gets a five-minute browser TTL.
- CloudFront compresses on the fly; nothing is pre-compressed.

A manual deploy is the same two syncs plus an invalidation; the exact commands, the resource creation steps, and the distribution, certificate, Route 53, bucket policy, and IAM details are in the React section of docs/DEPLOY.md. Resource IDs are documented there once and are not repeated here.

8. Checklist for a change

Use this before marking any site change done.

Step	Do
1	Make the change in the edition you started with.
2	If the file is shared (section 4), you are done with content. If it is duplicated, make the matching change in the other three editions.
3	If the React behavior changed, add or update the test that proves it. Follow the conventions in section 6.
4	Run `npm test` and `npm run build` in `react/` and make both green. Warnings in the build are errors in review.
5	Preview the editions locally: `python3 -m http.server 8080` at the root, `dotnet run --project blazor/src/Portfolio.Blazor`, and `npm start` in `angular/` and in `react/`.
6	Check the cross-links survived: the edition menu in the header and the four About chips, identical on every edition, with their hints; the hero highlights stay plain.
7	Never add a `style` attribute, a CSS-in-JS library, or an inline `<script>`; the CSP blocks them in production. New rules go in `react.css`; a color change edits its palette block, and the other three palette files if it is meant to be shared.
8	Commit all editions together. No AI attribution trailers in the message.
9	Push to `main` only when asked. The workflow tests, builds, and deploys every edition.

9. Links

What	Where
Live, React edition	https://react.mvcprogrammer.com
Live, Angular edition	https://angular.mvcprogrammer.com
Live, Blazor edition	https://blazor.mvcprogrammer.com
Live, vanilla edition	https://mvcprogrammer.com
Repository	https://github.com/mvcprogrammer/mvcprogrammer.com
React project	react/
React app source	react/src/app
React README	react/README.md
Angular guideline	docs/ANGULAR-EDITION.pdf
Blazor guideline	docs/BLAZOR-EDITION.pdf
Deploy walkthrough	docs/DEPLOY.md
Deploy workflow	.github/workflows/deploy.yml
Experience API spec	api/openapi.yaml
Contact Lambda	infra/lambda/contact/index.mjs
Workflow runs	GitHub Actions